

Jest do zero

Do Codespace ao primeiro projeto de testes

O QUE VOCÊ VAI APRENDER

Criar um projeto JavaScript no Codespace, instalar o Jest e preparar o ambiente para executar testes automáticos.

PASSO 1

Abra o Codespace

- Entre no GitHub e abra o repositório indicado para a atividade.
- Clique em Code -> Codespaces -> Create codespace on main.
- Espere o editor abrir no navegador. À esquerda ficam os arquivos; na parte inferior fica o terminal.

Terminal - confira o ambiente

```
1 $ node --version
2 $ npm --version
3 $ pwd
```

CONFIRA

Se os dois primeiros comandos exibirem números de versão, o ambiente está pronto para continuar.

PASSO 2

Crie o projeto e instale o Jest

Terminal - projeto aula-jest

```
1 $ mkdir aula-jest
2 $ cd aula-jest
3 $ npm init -y
4 $ npm install --save-dev jest
```

- mkdir cria a pasta e cd entra nela.
- npm init -y cria o arquivo package.json.
- --save-dev registra o Jest como ferramenta de desenvolvimento.

PASSO 3

Configure o comando de teste

package.json - trecho importante

```
1 "scripts": {
2   "test": "jest"
3 }
```

ANTES DE CONTINUAR

Salve o arquivo. No terminal, confirme que você ainda está dentro da pasta aula-jest.

Seu primeiro teste

Números, PASS, FAIL e execução individual

PASSO 4

Crie a função e o arquivo de teste

```
soma.js
1 function somar(a, b) {
2   return a + b;
3 }
4
5 module.exports = somar;
```

```
soma.test.js
1 const somar = require('./soma');
2
3 test('2 + 3 deve ser 5', () => {
4   expect(somar(2, 3)).toBe(5);
5 });
```

COMO LER O TESTE

- `test(...)` cria e descreve um teste.
- `expect(...)` recebe o valor real que será verificado.
- `.toBe(5)` informa o valor exato esperado.

PASSO 5

Execute e interprete o resultado

```
Terminal - execute todos os testes
1 $ npm test
2 PASS ./soma.test.js
3 Tests: 1 passed, 1 total
```

PASS

O resultado real foi igual ao esperado. O teste passou.

EXPERIMENTE UM FAIL

Troque `toBe(5)` por `toBe(10)` e rode novamente. Compare Expected com Received. Depois, volte para 5.

PASSO 6

Execute somente o teste que deseja

TODOS

```
$ npm test
```

Todos os arquivos `.test.js`

UM ARQUIVO

```
$ npx jest soma.test.js
```

Somente `soma.test.js`

UM TESTE

```
$ npx jest -t "2 + 3"
```

Filtra pelo nome do teste

Dois novos usos

Além de contas: teste de texto e teste de verdadeiro/falso

PASSO 7

Crie duas funções simples

```
utilidades.js

1 function saudacao(nome) {
2   return 'Olá, ' + nome + '!';
3 }
4 function maiorDeIdade(idade) {
5   return idade >= 18;
6 }
7 module.exports = { saudacao, maiorDeIdade };
```

USO 1 | TEXTO

Compare palavras e frases. Pontuação e acentos também fazem parte do resultado.

USO 2 | VERDADEIRO / FALSO

Uma regra pode responder sim ou não. Em JavaScript, usamos true e false.

```
utilidades.test.js

1 const { saudacao, maiorDeIdade } =
2   require('./utilidades');
3
4 test('cumprimenta Ana', () => {
5   expect(saudacao('Ana'))
6     .toBe('Olá, Ana!');
7 });
```

```
continue em utilidades.test.js

1 test('18 é maior de idade', () => {
2   expect(maiorDeIdade(18))
3     .toBe(true);
4 });
5 test('17 não é maior', () => {
6   expect(maiorDeIdade(17))
7     .toBe(false);
8 });
```

PASSO 8

Pratique e observe

1 Salve

Salve os dois arquivos no mesmo diretório.

2 Execute

Use `npx jest utilidades.test.js`.

3 Faça falhar

Troque Ana por Bia apenas no valor esperado.

4 Corrija

Leia Expected e Received; depois corrija o teste.

O que você já consegue fazer

- ☐ Executar todos os testes ou apenas um arquivo.
- ☐ Comparar números, textos e valores true/false.
- ☐ Usar a mensagem de FAIL para encontrar uma diferença.

Terminou? Tente este.

Desafio extra: par ou ímpar

SUA MISSÃO

A função `ehPar` já está pronta. Crie sozinho o arquivo `ehPar.test.js` e escreva os testes que comprovam seu funcionamento.

PASSO 9

Crie o arquivo da função

```
1 function ehPar(numero) {  
2   return numero % 2 === 0;  
3 }  
4  
5 module.exports = ehPar;
```

O operador `%` calcula o resto de uma divisão. Um número par deixa resto zero quando é dividido por 2.

NÃO CRIE OUTRO `numero.js`

Copie essa função como está. Seu trabalho principal será criar o arquivo de teste.

PASSO 10

Implemente `ehPar.test.js`

- Importe a função com `require('./numero')`.
- Crie um teste chamado 4 é par e verifique se o resultado é `true`.
- Crie outro teste chamado 5 não é par e verifique se o resultado é `false`.
- Use as três estruturas estudadas: `test`, `expect` e `toBe`.

ehPar.test.js - ponto de partida

```
1 const ehPar = require('./numero');  
2  
3 // Escreva aqui o teste do número 4  
4  
5  
6 // Escreva aqui o teste do número 5  
7  
8
```

PASSO 11

Execute e confira

Terminal - somente o desafio

```
1 $ npx jest ehPar.test.js
```

Seu arquivo passou?

- ☐ Os dois testes aparecem no resultado.
- ☐ O Jest mostra 2 passed e 2 total.
- ☐ O nome dos testes explica o que foi verificado.